MACHINE LEARNING / OPTIMIZATION

Gradient Descent
Batch → Stochastic → Mini-Batch

A complete tutorial, from zero to production level

Intuition | The math | Code from scratch | Visualizations | Scikit-learn | Interview traps

Dark theme edition · read it, then re-type every code block by hand

What is inside

1. **The ground floor** — what we are actually minimizing
2. **Batch Gradient Descent** — recap, and its two fatal problems
3. **Stochastic Gradient Descent (SGD)** — the idea, the math, the code
4. **Epochs vs updates** — the question that confuses everyone
5. **Why "stochastic"?** — randomness, instability, and why it is a feature
6. **When SGD is the only option** — big data & non-convex loss functions
7. **Learning schedules** — fixing the wobble at the end
8. **SGDRegressor in scikit-learn** — every parameter explained
9. **Mini-Batch Gradient Descent** — the one everyone actually uses
0. **Final comparison, cheat sheet & practice plan**

> **HOW TO READ THIS**
> Sections 1–2 are the basics. Sections 3–6 are the core. Sections 7–9 are the "pro" part (schedules, sklearn internals, mini-batch). Nothing here depends on anything you have not already read earlier in the document.

1. The ground floor

LEVEL: BEGINNER

Before anything else, be very clear about what gradient descent is *for*. Every supervised ML model has (a) some parameters, and (b) a loss function that scores how wrong those parameters are. Training a model = finding the parameter values that make the loss as small as possible. Gradient descent is just a procedure for doing that search.

1.1 The model

Take linear regression with *n* input columns:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

β_0 is the intercept (bias), $\beta_1 \ldots \beta_n$ are the coefficients (weights)

So with 5 input columns you have **6 numbers to learn**: five coefficients plus one intercept. Remember that "+1", it comes back in the cost calculations later.

1.2 The loss function

For regression we use Mean Squared Error over the *m* rows of the dataset:

$$L(\beta) = (1/m) \cdot \Sigma_{i=1..m} (y_i - \hat{y}_i)^2$$

m = number of rows. Squared so that errors do not cancel out and so it stays differentiable.

1.3 The update rule

The derivative of the loss with respect to a parameter tells you which way that parameter must move to make the loss *bigger*. So you step in the opposite direction — that minus sign is the whole algorithm:

$$\beta_j \leftarrow \beta_j - \eta \cdot \partial L / \partial \beta_j$$

η (eta) = learning rate, the step size. Repeat for every parameter, over and over.



One parameter, one loss curve. Start anywhere, follow the slope downhill, and the steps automatically get smaller as the slope flattens near the minimum.

1.4 Three words you must not mix up

Term	Meaning
Epoch	One full pass over the entire training dataset.
Update	One application of the rule above — the parameters actually change value once.
Batch size	How many rows you look at before performing one update.

THE ONE IDEA BEHIND THIS WHOLE DOCUMENT

Batch GD, Stochastic GD and Mini-batch GD are **not three different algorithms**. They are the *same* algorithm with three different batch sizes: batch size = m (all rows), batch size = 1 (one row), batch size = b (a small group). Everything else — the formulas, the code, the learning rate — is identical.

2. Batch Gradient Descent, and why it breaks

LEVEL: BEGINNER → INTERMEDIATE

In Batch Gradient Descent (also called Vanilla GD) you look at **the complete dataset**, compute the derivative of the loss over all rows, and only then update the parameters — **once per epoch**.

2.1 The math

For linear regression the two derivatives are:

$$\partial L/\partial \beta_0 = -(2/m) \cdot \Sigma_{i=1..m} (y_i - \hat{y}_i)$$

$$\partial L/\partial \beta_j = -(2/m) \cdot \Sigma_{i=1..m} (y_i - \hat{y}_i) \cdot x_{ij}$$

x_{ij} = value of column j in row i. Note the Σ running over **all m rows** — that is the expensive part.

2.2 The code (the version you already wrote)

```
class GDRegressor:

    def __init__(self, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self, X_train, y_train):
        # start from a neutral guess
        self.intercept_ = 0
        self.coef_ = np.ones(X_train.shape[1])

        for i in range(self.epochs):
            # prediction for ALL rows at once (vectorised)
            y_hat = np.dot(X_train, self.coef_) + self.intercept_

            # derivative of intercept, over ALL rows
            intercept_der = -2 * np.mean(y_train - y_hat)
            self.intercept_ = self.intercept_ - (self.lr * intercept_der)

            # derivative of every coefficient, over ALL rows
            coef_der = -2 * np.dot((y_train - y_hat), X_train) / X_train.shape[0]
            self.coef_ = self.coef_ - (self.lr * coef_der)

    def predict(self, X_test):
        return np.dot(X_test, self.coef_) + self.intercept_
```

This works, it is smooth, and on a small dataset it is great. Now the bad news.

2.3 Problem 1 — the number of derivatives explodes

In one epoch, for **one** parameter, you sum over all *m* rows. You have (columns + 1) parameters. And you do all of that once per epoch. So:

$$\text{total derivative terms} = m \times (n+1) \times \text{epochs}$$

Put real numbers in it and watch it detonate:

Rows (m)	Columns (n)	Parameters (n+1)	Epochs	Derivative terms computed
1,000	5	6	15	90,000
10,000	20	21	100	21,000,000
100,000 (10^5)	100 (10^2)	~10^2	1,000 (10^3)	**10^{10}** — ten billion

WHY THIS MATTERS

Ten billion terms on a single dataset, and that dataset is not even big by modern standards. The two places deep learning actually lives — images and text — produce datasets vastly larger than this. Batch GD simply does not finish in reasonable time there.

2.4 Problem 2 — the RAM wall

Look at this line again:

```
y_hat = np.dot(X_train, self.coef_) + self.intercept_
```

This is **vectorization**: instead of a Python loop over rows, NumPy does one big matrix-vector multiplication. That is a huge speed win — but it has a condition attached. To multiply that matrix, **the entire X_train must be loaded into RAM at once**.

If your training data is 40 GB and your machine has 16 GB of RAM, the operation is not slow — it is *impossible*. You get a `MemoryError` and the process dies before a single epoch completes.

Pain point	Root cause	Consequence
Very slow on large data	Sum over all m rows for every parameter, every epoch	Training takes hours or days
Crashes on large data	Vectorization needs the whole matrix in memory	MemoryError; cannot train at all on a modest machine
Stuck in local minima	The update path is perfectly smooth, no noise	On non-convex losses it settles in the first valley it finds (section 6)

SO WHAT DO WE CHANGE?

Notice that the *maths* of gradient descent is not the problem. The **implementation** is. We are choosing to look at every row before updating. Nobody forced us to. Change that one choice and both problems disappear. That change is called **Stochastic Gradient Descent**.

3. Stochastic Gradient Descent

LEVEL: CORE

The change is almost embarrassingly small:

Batch Gradient Descent	Stochastic Gradient Descent
Look at **all m rows** → compute one average gradient → update once. 1 epoch = 1 update.	Look at **one randomly chosen row** → compute the gradient from just that row → update. Then pick another row, update again… 1 epoch = m updates.

That is it. You stop waiting for the full picture and start acting on partial information immediately. Instead of one careful, well-informed step per epoch, you take *m* quick, roughly-correct steps.

3.1 The math (what actually drops out of the formula)

Start from the batch formula and set m = 1, because we are using exactly one row. Two things vanish: the summation Σ, and the division by m.

Batch

$$\partial L/\partial \beta_0 = -(2/m)\Sigma(y_i - \hat{y}_i)$$

$$\partial L/\partial \beta_j = -(2/m)\Sigma(y_i - \hat{y}_i)x_{ij}$$

Stochastic

$$\partial L/\partial \beta_0 = -2(y_i - \hat{y}_i)$$

$$\partial L/\partial \beta_j = -2(y_i - \hat{y}_i)x_{ij}$$

where *i* is one randomly picked row index. Every quantity here is a **single number** (a scalar), not a vector of m values:

- `y[idx]` — one true value, a scalar
- `y_hat` — one prediction, a scalar
- `X_train[idx]` — one row, shape `(n,)` not `(m, n)`

So `np.dot(X_train[idx], coef_)` gives a scalar, and multiplying the scalar error by the row `X_train[idx]` of shape (n,) gives back a gradient vector of shape (n,) — exactly the shape of `coef_`. The shapes line up perfectly; that is your sanity check while coding.

> **SHAPE CHECK (do this every time)**
> With the diabetes dataset: `X_train.shape` is `(353, 10)`. Then `X_train[idx].shape` is `(10,)`, `coef_.shape` is `(10,)`, their dot product is a scalar, and `scalar * X_train[idx]` is again `(10,)`. If a shape surprises you, you picked the wrong axis somewhere.

3.2 SGD from scratch — the full class

Compare this line by line with the batch version in section 2.2. Only the inside of the epoch loop changed.

```python
import numpy as np

class SGDRegressor:

    def __init__(self, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs

    def fit(self, X_train, y_train):
        self.intercept_ = 0
        self.coef_ = np.ones(X_train.shape[1])

        for i in range(self.epochs):
            for j in range(X_train.shape[0]):     # <-- m updates per epoch

                # 1. pick ONE random row (this is the "stochastic" part)
                idx = np.random.randint(0, X_train.shape[0])

                # 2. predict for that single row -> a scalar
                y_hat = np.dot(X_train[idx], self.coef_) + self.intercept_

                # 3. intercept derivative: no summation, no /m
                intercept_der = -2 * (y_train[idx] - y_hat)
                self.intercept_ = self.intercept_ - (self.lr * intercept_der)

                # 4. coefficient derivatives: scalar error x one row
                coef_der = -2 * np.dot((y_train[idx] - y_hat), X_train[idx])
                self.coef_ = self.coef_ - (self.lr * coef_der)

        print(self.intercept_, self.coef_)

    def predict(self, X_test):
        return np.dot(X_test, self.coef_) + self.intercept_
```

Running it on the diabetes dataset:

```python
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score

X, y = load_diabetes(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=2)

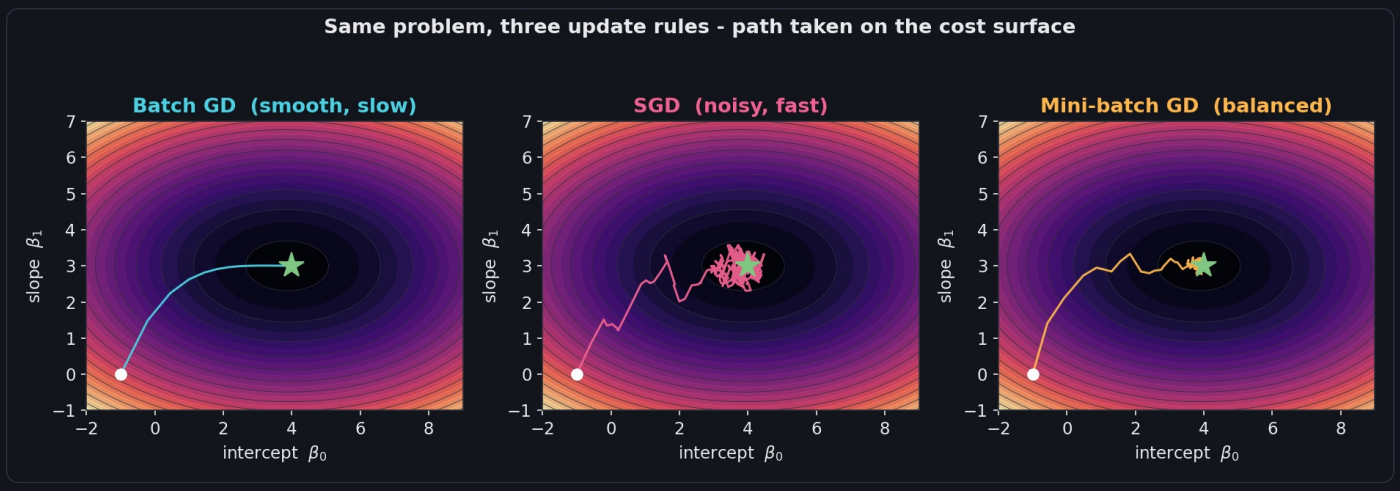
sgd = SGDRegressor(learning_rate=0.01, epochs=40)
sgd.fit(X_train, y_train)

y_pred = sgd.predict(X_test)
print(r2_score(y_test, y_pred))     # ~0.43, with only 40 epochs
```

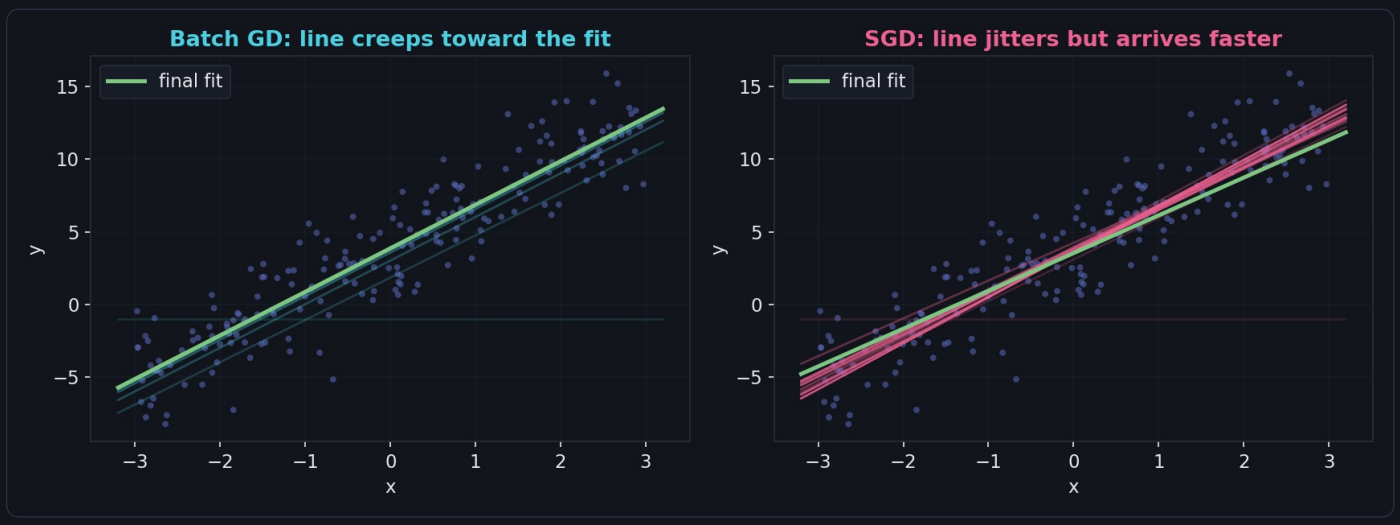
TUNING NOTE FROM THE EXPERIMENT

With a very small learning rate SGD looks bad — it needs many more epochs. Raise the learning rate (e.g. 0.01 → 0.05) and the same number of epochs gives a clearly better R^2. Learning rate and epochs always have to be tuned together; never judge an optimizer from one setting.

3.3 What the two algorithms look like while running



The coloured surface is the cost function (dark = low cost). White dot = starting point, green star = optimal parameters. Batch GD walks a clean curve. SGD staggers, because each step is based on one random row. Mini-batch (section 9) sits in between.



The same runs seen as the regression line itself. Faint lines are intermediate states, the green line is the final fit. Batch GD's line slides across smoothly; SGD's line jumps around but locks onto the data in far fewer passes.

3.4 The two gifts and the one curse

Gift 1 — much faster convergence (in epochs)

More updates per epoch means you reach a good solution in fewer passes over the data. Something that takes batch GD 50 epochs may take SGD 5.

Gift 2 — almost no memory needed

You only ever hold **one row** in memory at a time. It does not matter if the dataset is 1 GB, 100 GB or streaming off a disk — the RAM requirement stays flat. The MemoryError from section 2.4 can never happen.

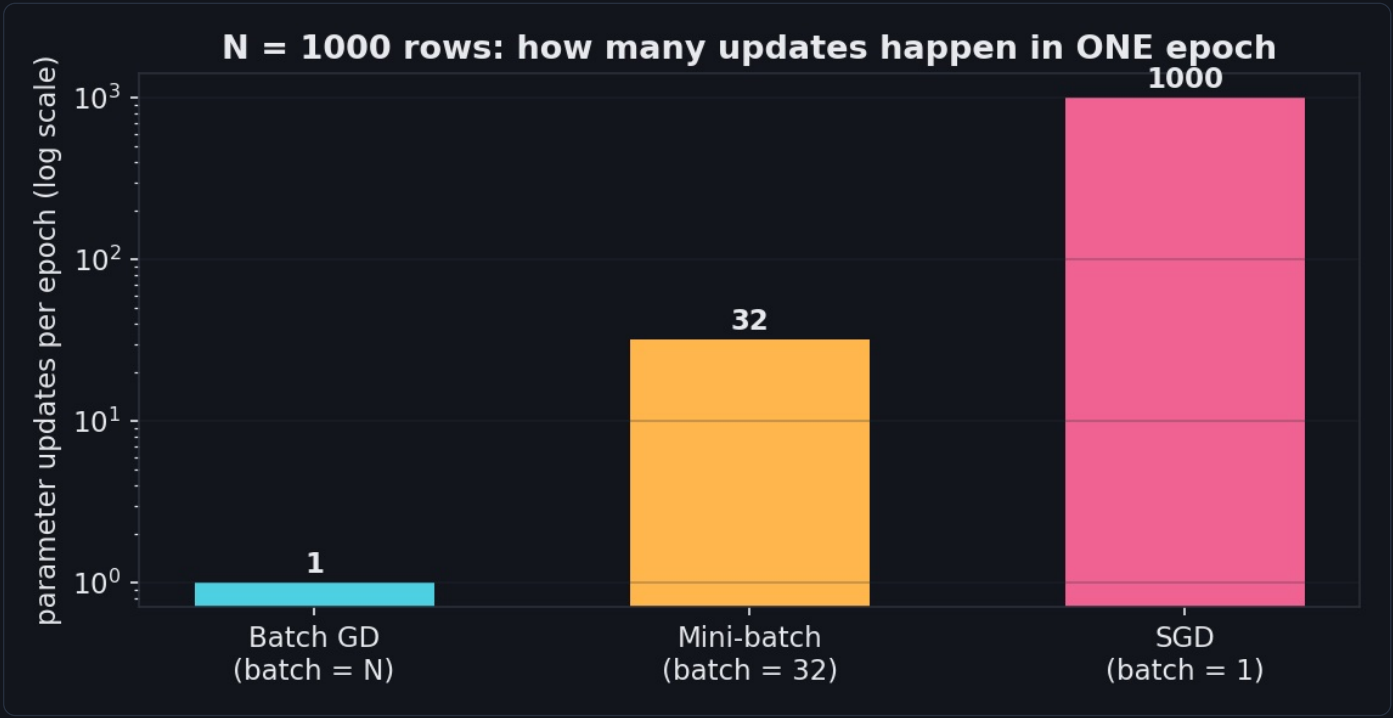
Curse — the answer is not stable

Run batch GD twice on the same data and you get identical numbers. Run SGD twice and you get **different numbers each time**, because the rows are drawn at random. The results will be close to each other and close to the true optimum, but they are not the same, and they are not guaranteed to be the exact optimum.

	Batch GD	Stochastic GD
Rows used per update	all m	1 (random)
Updates per epoch	1	m
Epochs needed	many	few
Memory	whole dataset in RAM	one row
Path to the minimum	smooth, always improving	noisy, sometimes worse before better
Reproducible result	yes	no (unless you fix the random seed)
Good for	small, clean datasets	large data, non-convex losses

4. Epochs vs updates — the question that trips people up

LEVEL: INTERMEDIATE · CLASSIC INTERVIEW TRAP



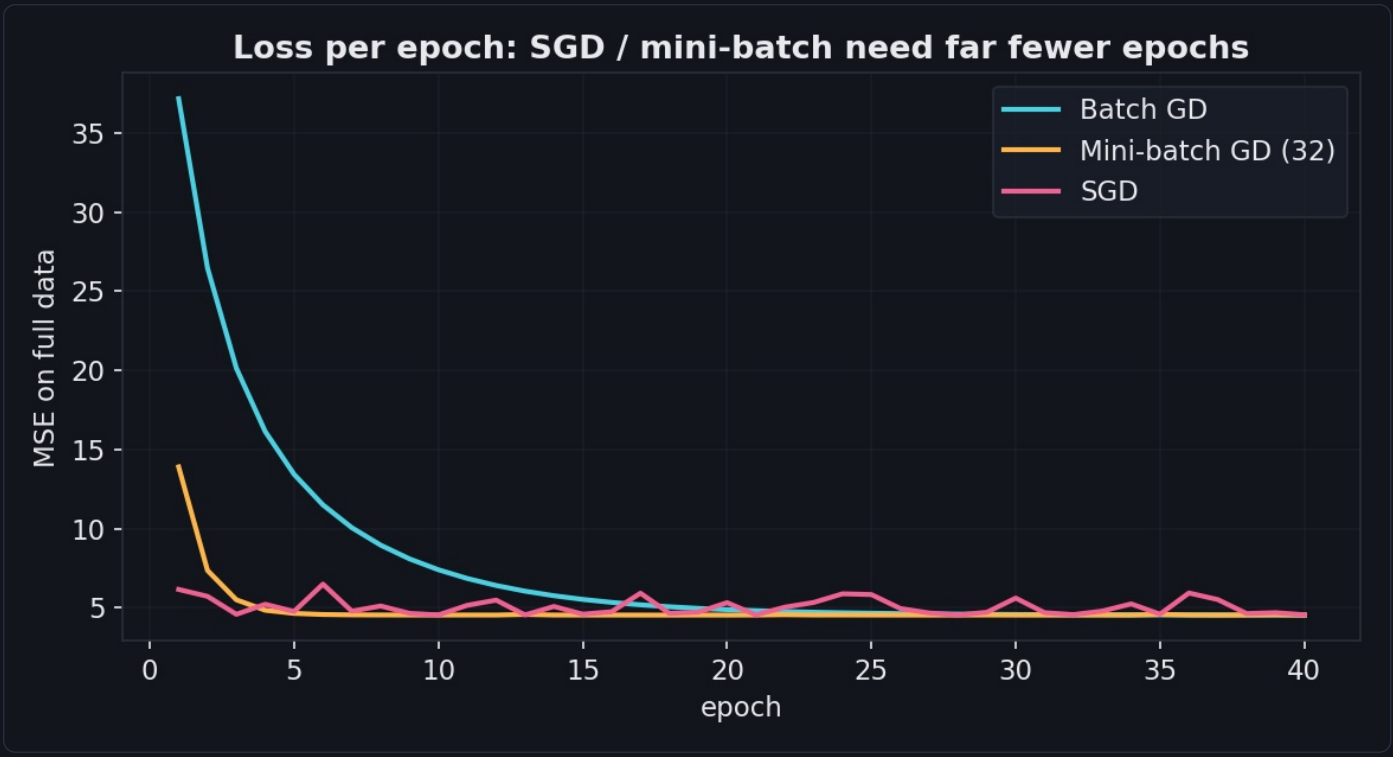
With 1000 rows, one epoch means one update for batch GD, 32 updates for mini-batch (batch size 32), and 1000 updates for SGD.

THE QUESTION

"If batch GD and SGD are given the **same number of epochs**, which one finishes faster?" — Pause and answer before reading on.

Answer: batch gradient descent is faster for an equal number of epochs. With 100 epochs, batch GD performs 100 updates. SGD performs 100 × m updates — with m = 1000 that is 100,000 updates, each carrying its own Python-level overhead. Per epoch, SGD does strictly more work.

The reason SGD wins in practice is that **it does not need the same number of epochs**. It needs far fewer. The comparison that matters is "time to reach a good solution", not "time per epoch".



Loss after each epoch. Batch GD is still descending when SGD and mini-batch have already flattened out near the optimum.

4.1 Measure it yourself

```
import time

start = time.time()
sgd = SGDRegressor(learning_rate=0.01, epochs=40)
sgd.fit(X_train, y_train)
print("time taken:", time.time() - start)
```

Do this on the diabetes dataset (353 training rows) and you will find something uncomfortable: **SGD takes longer than batch GD**. That is not a bug, and it does not contradict anything above. Three reasons:

1. The dataset is tiny, so batch GD's "expensive" full-data pass is one cheap vectorized NumPy call.
2. SGD replaces that single fast C-level operation with an inner Python loop running m times per epoch — Python loops are slow.
3. Both were given the same number of epochs, which, as we just established, is the setting that favours batch GD.

DO NOT OVER-GENERALISE FROM SMALL DATA

"SGD is faster" is a claim about **large** datasets and about **time-to-convergence**. On a 353-row toy dataset, batch GD wins on the clock. On 10 million rows that do not fit in memory, batch GD does not produce an answer at all.

5. Why "stochastic"? The randomness, in detail

LEVEL: INTERMEDIATE

Google the word and you get: *randomly determined; having a random probability distribution that may be analysed statistically but not predicted precisely.* That is exactly what the algorithm does.

In the inner loop, you do **not** walk through rows in order 0, 1, 2, 3... You draw an index at random each time:

```
idx = np.random.randint(0, X_train.shape[0])
```

So a row can be picked twice in one epoch and another row skipped entirely. The "epoch" here means "m updates", not "every row seen exactly once".

5.1 Consequence: your answer changes every run

```
sgd.fit(X_train, y_train)
# run 1 -> intercept 152.1 , coef_ [ 45.2, -78.4, ... ]
# run 2 -> intercept 149.8 , coef_ [ 43.9, -81.1, ... ]   different!
```

Close, useful, not identical. If you need reproducibility, fix the seed with `np.random.seed(42)` (or pass `random_state` to scikit-learn's implementation).

5.2 Consequence: the path is not monotonically improving

With batch GD, every step is guaranteed to be at least as good as the last (for a sensible learning rate). With SGD, step *t+1* can genuinely be **worse** than step *t*, because that particular random row pulled the parameters in a misleading direction. Zoom out over a few hundred updates and the trend is still downward — it just hesitates on the way.

> **THE MENTAL PICTURE**
>
> Batch GD is a careful person who reads the whole map before each step. SGD is someone who glances at one signpost and walks. The careful person never goes the wrong way but takes forever. The other one occasionally walks into a wall, yet arrives first — and, as the next section shows, is the only one who can climb out of a dead end.

6. When SGD is not just faster, but necessary

LEVEL: ADVANCED

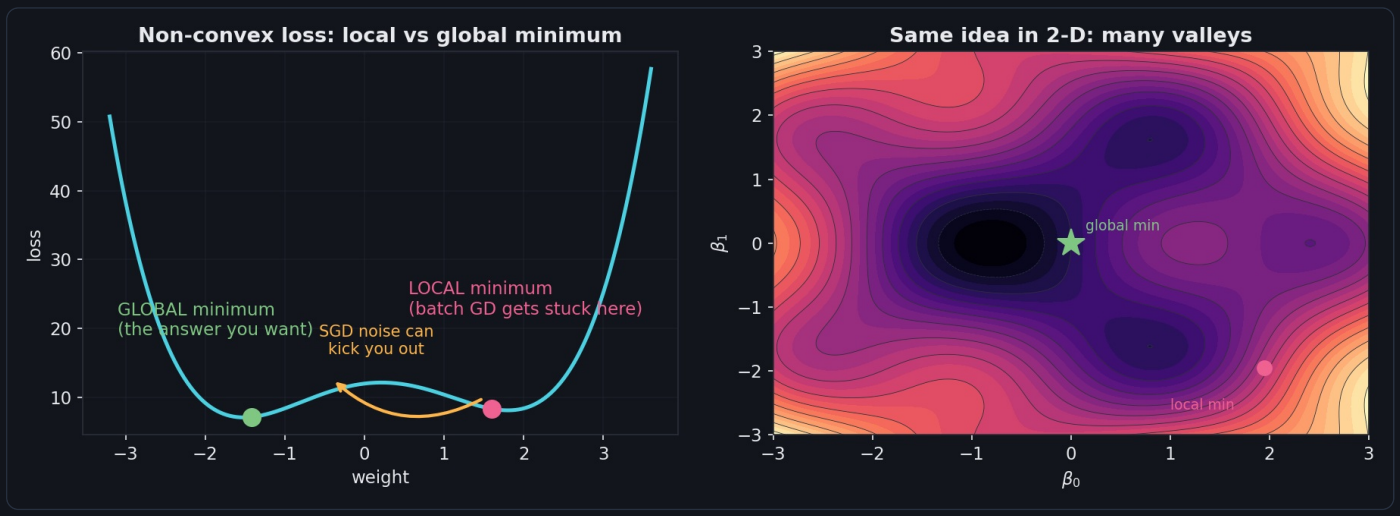
6.1 Case A — big data

Many rows, many columns, or both. Batch GD either takes impractically long or runs out of RAM. SGD converges in fewer epochs and needs memory for one row. This is why SGD and its mini-batch variant are the default in deep learning, where the two dominant data types — images and text — produce enormous datasets.

6.2 Case B — non-convex loss functions

First, the definition. A function is **convex** if, for any two points on its graph, the straight line segment joining them never dips below the function. Pick any two points on a bowl, connect them, and the line stays above the surface. If the function ever cuts above that line — it is **non-convex**.

Convex loss	Non-convex loss
Exactly one minimum. Local minimum = global minimum. Wherever you start, gradient descent reaches the same best answer. Example: linear regression with MSE.	Many valleys. There is one global minimum plus several local minima that are worse but still look like a bottom to the algorithm. Examples: logistic regression variants, SVMs with some losses, nearly every neural network.



Left: the danger. If you start on the right side, batch GD slides smoothly into the local minimum and stops there forever, reporting a worse-than-optimal solution. Right: the same landscape as a contour map — dark regions are low cost, and there is more than one of them.

Here the smoothness of batch GD becomes a liability. It descends politely into the nearest valley and stops. Your final parameters are wrong — not because the maths failed, but because of where you happened to start.

SGD's noise is the escape hatch. Because each update is based on one random row, the parameters get knocked around. A big enough random kick can throw the parameters **out of a local minimum**, after which the search continues and may find the global minimum.

THE TRADE-OFF, STATED HONESTLY

SGD *can* escape local minima; it is not guaranteed to. It depends on the learning rate: too small and the kicks are too weak to escape, too large and it never settles anywhere. This is exactly why learning-rate tuning matters more for SGD than for batch GD — and why the next section exists.

7. The wobble problem and learning schedules

LEVEL: ADVANCED

There is one more flaw in SGD. It converges quickly to the neighbourhood of the solution — and then it never stops moving. Each new random row yanks the parameters a little this way, a little that way. You end up permanently bouncing *around* the optimum instead of settling *on* it.

The fix is to stop treating the learning rate as a constant. Take big steps early (when you are far away and want to cover ground) and progressively smaller steps later (when you are close and want to settle). This is called a **learning schedule**, or learning-rate decay.

$$\eta(t) = t_0 / (t + t_1) \quad \text{where} \quad t = (\text{epoch} \times m) + i$$

t is the global update counter. t_0 and t_1 are constants you choose.

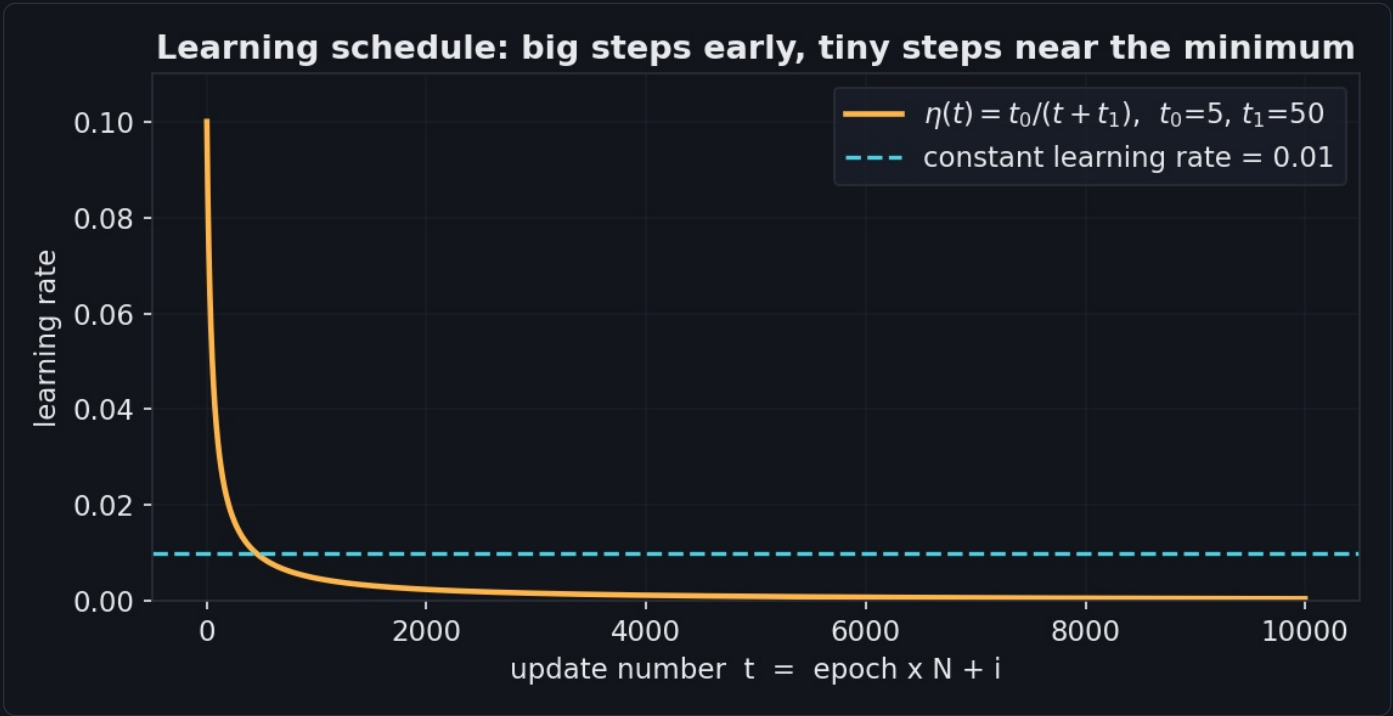
```
t0, t1 = 5, 50

def learning_schedule(t):
    return t0 / (t + t1)

# inside the fit loop:
for i in range(self.epochs):
    for j in range(X_train.shape[0]):
        idx = np.random.randint(0, X_train.shape[0])
        ...
        lr = learning_schedule(i * X_train.shape[0] + j)   # <-- varies every update
        self.coef_ = self.coef_ - (lr * coef_der)
        self.intercept_ = self.intercept_ - (lr * intercept_der)
```

Worked example with m = 100 rows and t_0=5, t_1=50:

Moment	t	Learning rate η(t)
very first update	0	5 / 50 = **0.100**
end of epoch 1	100	5 / 150 = **0.033**
end of epoch 5	500	5 / 550 = **0.009**
end of epoch 50	5000	5 / 5050 = **0.001**



The step size shrinks automatically as training proceeds. Early updates explore aggressively; late updates barely move, which lets the parameters settle instead of orbiting.

PRACTICAL EFFECT

Adding this one function to the from-scratch SGD class noticeably improves the final R^2 on the same data with the same number of epochs. This particular $1/(t+c)$ form is the simplest of many; exponential decay, step decay and cosine decay all follow the same principle. Learning schedules matter far more in deep learning than in linear regression, but the concept is the same everywhere — so learn it here, where it is simple.

8. SGDRegressor in scikit-learn

LEVEL: PRACTICAL

You will rarely ship your own class. In production you use scikit-learn's `SGDRegressor` — a small class that implements stochastic gradient descent internally.

```
from sklearn.linear_model import SGDRegressor

reg = SGDRegressor(
        max_iter=100,
        learning_rate='constant',
        eta0=0.01,
        penalty=None,
        random_state=42)

reg.fit(X_train, y_train)
y_pred = reg.predict(X_test)
print(r2_score(y_test, y_pred))
print(reg.intercept_, reg.coef_)
```

8.1 Why this class is more useful than it looks

Its real strength is that it is **not tied to linear regression**. Remember that a machine learning algorithm is largely "a loss function + a way to minimize it". Keep SGD as the minimizer and swap the loss, and you get a different algorithm:

loss=	What you get
`'squared_error'` (default)	Ordinary linear regression
`'huber'`	Regression that is robust to outliers
`'epsilon_insensitive'`	Support Vector Regression (linear SVR)
`'squared_epsilon_insensitive'`	SVR variant with squared penalty

8.2 Every parameter that matters

Parameter	What it controls
`loss`	Which loss function to minimize — see the table above. This is what turns the same optimizer into a different algorithm.
`penalty`	Regularization: `'l2'` (ridge, default), `'l1'` (lasso), `'elasticnet'`, or `None`. Set to `None` when you want plain unregularized regression.
`alpha`	Strength of that regularization. Bigger = more shrinkage of coefficients.
`l1_ratio`	Mix between L1 and L2 when `penalty='elasticnet'`.
`fit_intercept`	Whether to learn β_0. Leave it `True` unless your data is already centred.
`max_iter`	Maximum number of epochs.
`tol`	Early-stopping tolerance. If the improvement between iterations is smaller than this, training stops — the parameters are no longer meaningfully improving, so why keep going.
`shuffle`	Reshuffle the training data after each epoch. Keep it `True`.
`random_state`	Fixes the randomness. Without it, every run gives slightly different coefficients (section 5.1). With it, results are reproducible.
`learning_rate`	The schedule: `'constant'` (never changes, uses `eta0`), `'optimal'`, `'invscaling'` (the 1/t style decay of section 7), `'adaptive'` (drop the rate only when the loss stops improving).
`eta0`	The initial learning rate. With `'constant'` this is the rate, permanently.
`power_t`	Exponent used by `'invscaling'` to control how fast the rate decays.
`early_stopping`	Hold out part of the training data and stop when validation score stops improving.
`warm_start`	Continue from the previously fitted coefficients instead of restarting.

EXPECT SMALL DIFFERENCES

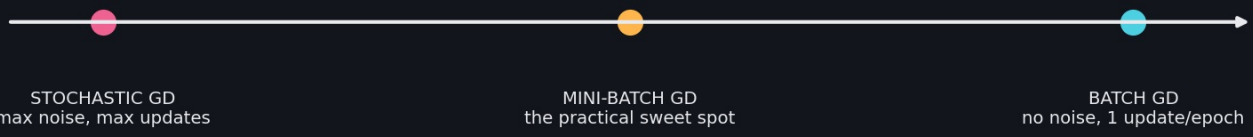
Your from-scratch class and `SGDRegressor` will not print identical numbers on the same data — sklearn's internals (default regularization, schedule, stopping rule) differ from your simple loop. Either can score slightly higher on a given dataset. Writing your own version is for *understanding*; shipping sklearn's is for *work*.

9. Mini-Batch Gradient Descent

LEVEL: CORE · THIS IS THE ONE YOU WILL ACTUALLY USE

Batch GD is accurate but slow and memory-hungry. SGD is fast and light but noisy and unstable. Mini-batch takes the middle road: update after seeing a **small group of rows** — not all of them, not just one.

All three algorithms are ONE algorithm with a different batch size



STOCHASTIC GD
max noise, max updates

MINI-BATCH GD
the practical sweet spot

BATCH GD
no noise, 1 update/epoch

Batch size is a dial, not a category. Turn it to 1 and you have SGD; turn it to m and you have batch GD; leave it at 32 or 64 and you have mini-batch GD.

9.1 The arithmetic of batch size

$$\text{updates per epoch} = \lceil\, m \,/\, \text{batch_size} \,\rceil$$

Rows (m)	batch_size	Updates per epoch	This is called
100	100	1	Batch GD
100	10	10	Mini-batch GD
100	1	100	Stochastic GD
1,000	32	32	Mini-batch GD (typical)

So the two algorithms from earlier are just the two extreme settings of this one dial. Typical batch sizes in practice are powers of two — 32, 64, 128, 256 — chosen to fit comfortably in memory.

9.2 Mini-batch from scratch

Only two things change versus SGD: a `batch_size` argument, and picking a *list* of random indices instead of one.

```python
import random
import numpy as np

class MBGDRegressor:

    def __init__(self, batch_size, learning_rate=0.01, epochs=100):
        self.coef_ = None
        self.intercept_ = None
        self.lr = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size          # <-- new

    def fit(self, X_train, y_train):
        self.intercept_ = 0
        self.coef_ = np.ones(X_train.shape[1])

        for i in range(self.epochs):

            # how many updates in this epoch = rows / batch_size
            for j in range(int(X_train.shape[0] / self.batch_size)):

                # pick batch_size DISTINCT random row indices
                idx = random.sample(range(X_train.shape[0]), self.batch_size)

                # fancy indexing -> a mini matrix of shape (batch_size, n)
                y_hat = np.dot(X_train[idx], self.coef_) + self.intercept_

                intercept_der = -2 * np.mean(y_train[idx] - y_hat)
                self.intercept_ = self.intercept_ - (self.lr * intercept_der)

                coef_der = -2 * np.dot((y_train[idx] - y_hat), X_train[idx])
                self.coef_ = self.coef_ - (self.lr * coef_der)

        print(self.intercept_, self.coef_)

    def predict(self, X_test):
        return np.dot(X_test, self.coef_) + self.intercept_
```

```python
mbr = MBGDRegressor(batch_size=int(X_train.shape[0] / 10), learning_rate=0.01, epochs=100)
mbr.fit(X_train, y_train)
y_pred = mbr.predict(X_test)
print(r2_score(y_test, y_pred))    # ~0.45 ; tune lr / epochs / batch_size to push it up
```

THREE THINGS TO NOTICE IN THAT CODE

1. `random.sample` draws *distinct* indices, so no row is used twice in the same mini-batch (unlike `np.random.randint` in SGD).
2. `X_train[idx]` with a list of indices is **fancy indexing** — it returns a new array of shape `(batch_size, n)`.
3. Because the mini-batch is a matrix again, `np.mean` and `np.dot` come back — you get vectorization benefits on a chunk small enough to always fit in memory. That is the real reason mini-batch wins on hardware.

9.3 What it looks like while running

Go back to the three-panel figure in section 3.3. The mini-batch path is recognisably the same shape as the batch GD path, but with a light tremble — not as smooth as batch, nowhere near as chaotic as SGD. Increase the batch size and the tremble shrinks; decrease it and the path gets wilder. You are literally dialling the amount of noise you want.

The end-of-training wobble from section 7 still exists, just smaller. The same fix applies: use a learning schedule, and the parameters settle instead of orbiting the optimum.

9.4 Mini-batch in scikit-learn (the workaround)

Here is an annoyance worth knowing: `SGDRegressor` has **no** `batch_size` **parameter**. Internally it uses batch size 1. To get mini-batch behaviour you drive it manually with `partial_fit`, which performs exactly one update on whatever rows you hand it:

```python
from sklearn.linear_model import SGDRegressor
import numpy as np, random

sgd = SGDRegressor(learning_rate='constant', eta0=0.1)

batch_size = 35
for i in range(100):                        # 100 epochs
    idx = random.sample(range(X_train.shape[0]), batch_size)
    sgd.partial_fit(X_train[idx], y_train[idx])   # one update on this mini-batch

print(sgd.intercept_, sgd.coef_)
print(r2_score(y_test, sgd.predict(X_test)))      # ~0.46
```

WHY partial_fit EXISTS

It is the streaming / out-of-core API: feed the model data in chunks it can hold in memory, one chunk at a time, forever. That is the same property that made SGD immune to the RAM wall in section 2.4 — here exposed as a public method. In Keras/PyTorch you do not need this workaround: `batch_size` is a first-class argument.

10. Final comparison, cheat sheet and practice plan

LEVEL: SUMMARY

10.1 The three side by side

	Batch GD	Stochastic GD	Mini-Batch GD
Rows per update	m (all)	1	b (e.g. 32)
Updates per epoch	1	m	m / b
Path	smooth, monotonic	very noisy	slightly noisy
Memory	whole dataset	one row	one batch
Vectorization	full, but risky	none (scalar work)	full, and safe
Escapes local minima	no	yes, often	yes, sometimes
Reproducible	yes	no	no
Use it when	data is small and convex	data is huge or streaming	**almost always**

10.2 Decision cheat sheet

Situation	Choose
Small dataset, convex loss, want the exact same answer every run	Batch GD
Dataset does not fit in RAM / arrives as a stream	SGD or mini-batch
Non-convex loss and you fear local minima	SGD (tune the learning rate carefully)
Deep learning, or any real-world tabular problem of decent size	Mini-batch (32 / 64 / 128)
Solution keeps bouncing around the optimum	Add a learning schedule
Answers differ between runs and that is a problem	Set `random_state` / `np.random.seed`

10.3 Mistakes people actually make

- **Judging an optimizer at one learning rate.** A bad η makes any method look useless. Sweep it.
- **Comparing methods at equal epochs.** Equal epochs favours batch GD by construction. Compare time-to-target instead.
- **Forgetting to remove the `/m` and the Σ** when moving from batch to SGD. The gradient then comes out scaled wrongly.
- **Not scaling the features.** Gradient descent on unscaled columns zig-zags badly. Standardize first.
- **Expecting SGD to hit the exact optimum.** It converges to a good neighbourhood; a learning schedule is what tightens it.
- **Leaving sklearn's default L2 penalty on** while comparing against your own unregularized class, then wondering why the coefficients differ.

10.4 Your practice plan

1. Re-type all three classes by hand, without copying. Derive the two derivative formulas on paper first.
2. Run each on the diabetes dataset and record R^2, epochs and wall-clock time in a table.
3. Sweep the learning rate over {0.001, 0.01, 0.05, 0.1} and watch which method is most sensitive to it.
4. Sweep the mini-batch size over {1, 8, 32, 128, m} and confirm that the extremes reproduce SGD and batch GD exactly.
5. Add the learning schedule to SGD and measure the improvement.
6. Repeat everything on a much larger synthetic dataset (`sklearn.datasets.make_regression` with 200,000 rows) — this is where the claims in section 6 finally show up on the clock.

> **ONE-LINE SUMMARY**
> Gradient descent is one algorithm with one knob. Turn the knob to "all rows" for accuracy, to "one row" for speed and noise, and to "a few rows" for the best of both — which is why mini-batch is what the world actually runs.

Master gradient descent here, in linear regression, and every optimizer you meet later — Momentum, RMSprop, Adam — is just this loop with a smarter step.